

**National University of Computer & Emerging
Sciences**
Karachi Campus



RetroCalc
Old-School Calculator

Project Proposal
Compiler Construction
Section: 6J

Group Members:
23k-2001 Muzammil Siddiqui
23k-0532 Rayyan Ali
23k-0568 Azaan Hussain

Language Concept

RetroCalc is a simple retro-style calculator programming language designed to perform arithmetic computations, variable storage, and basic program logic similar to early programmable calculators from the 1980s. The language is intended for beginners who want a simple scripting environment for mathematical calculations. The compiler will translate RetroCalc programs into an intermediate representation and execute them through an interpreter. The project demonstrates all phases of compiler construction including lexical analysis, parsing, semantic analysis, optimization, and execution.

Key Features

- Feature 1:
Support for integer and floating-point variables.
- Feature 2:
Basic arithmetic operations (+, −, *, /, %).
- Feature 3:
Variable assignment and usage.
- Feature 4:
Print statements to display results.
- Feature 5:
Simple control structures (if statements and loops).
- Feature 6:
Interactive REPL mode for testing expressions.
- Feature 7:
Basic optimizations like constant folding and dead code removal.

Example Program

Example RetroCalc program:

```
let x = 10
```

```
let y = 20
```

```
let sum = x + y
```

```
let product = x * y
```

```
print sum
```

```
print product
```

```
if sum > 20
```

```
  print sum + 5
```

```
end
```

Example with loop:

```
let i = 0
```

```
loop 5
```

```
  print i
```

```
  let i = i + 1
```

```
end
```

Target Output

Our compiler will produce:

- Three Address Code (Intermediate Representation)
- Optimized Intermediate Code
- Direct execution through interpreter
- Debug mode showing tokens and parse tree
- Interactive execution mode (REPL)

Output type:

- Interpreted language (execution through virtual machine style interpreter)
- Direct execution and display of results

Expected Challenges

- Challenge 1:
Designing a correct grammar that properly handles operator precedence.
- Challenge 2:
Building a symbol table that correctly tracks variables and scopes.
- Challenge 3:
Implementing optimizations like constant folding without breaking execution.
- Challenge 4 (optional):
Designing meaningful error messages with line numbers.

Bonus Features

Bonus features we plan to implement:

- Comments support:

`# this is a comment`

- Debug mode:

`compiler file.rc --debug`

- REPL mode:

`compiler --interactive`

- Error messages:

`Error at line 4: Undefined variable 'z'`